



Strings no C

Profa.: Andréia Rodrigues Casare

E-mail: casareandreia@gmail.com



Biblioteca <string.h>

- Essa biblioteca fornece várias funções para trabalhar com as strings no C.
- strlen, strcmp, strcat, strcpy, etc

Função strcmp

- Essa função compara duas strings segundo sua ordem alfabética e retorna um inteiro.
- Se for negativo, é porque a primeira string é menor que a segunda.
- Se for positivo, é porque a segunda string é maior que a primeira.
- Se retornar 0, é porque as **strings** são **idênticas**.
- Compara caractere por caractere começando do primeiro caractere até que os caracteres em ambas as strings sejam iguais ou um caractere NULL seja encontrado.
- Este processo continuará até que um caractere em qualquer string seja NULL ou os caracteres sejam desiguais.

Exemplo 1

```
#include <stdio.h>
#include <string.h>
int main(){
    char msg[10], msg2[10];
    int resu;
    printf("Digite a 1ª palavra: \n");
    fgets(msg,sizeof(msg),stdin);
    printf("Digite a 2ª palavra: \n");
    fgets(msg2,sizeof(msg2),stdin);
    resu=strcmp(msg,msg2);
    if (resu == 0){
        printf("as strings são iguais");
    }else{
        printf("as strings são diferentes");
    }
    return 0;
}
```

Exemplo 2

```
int main(){
    char str1[10], str2[10];
    int i, achou=2;
    printf("Digite a 1ª string: \n");
    fgets(str1,sizeof(str1),stdin);
    printf("Digite a 2ª string: \n");
    fgets(str2,sizeof(str2),stdin);
    for (i=0; str1[i] == str2[i];i++){
        if (str1[i] == 0 ){
            achou= 1;
        }
    }
    if (achou == 1){
        printf("\n As strings são iguais");
    }else{
        printf("\n As strings são diferentes");
    }
    return 0;
}
```

Exemplo 3

```
int main(){
    char aNomes[10][15], nome[15];
    int i, j, achou;
    printf("Digite os 10 nomes: \n");
    for (i=0; i < 10; i++){
        strupr(fgets(aNomes[i], sizeof(aNomes[i]), stdin));
    }
    while (1) {
        printf("Digite um nome para pesquisar ou (FIM para sair): \n");
        strupr(fgets(nome, sizeof(nome), stdin));
        if (strcmp (nome, "FIM\n") == 0){
            break;
        }
        achou=0;
        for (i=0; i < 10; i++){
            if (strcmp (nome, aNomes[i]) == 0){
                printf("%s foi encontrado no índice %d\n", nome, i);
                achou= 1;
                break;
            }
        }
        if (!achou){
            printf("%s não foi encontrado \n", nome);
        }
    }
}
```

Funções strcpy()

- Copia o conteúdo de uma string para outra.

- Sintaxe

- strcpy(destino, origem);

- Exemplo:

```
char nome1[20] = "Maria";
```

```
char nome2[20];
```

```
strcpy(nome2, nome1);
```

```
printf("%s", nome2);
```

Funções **strupr()** e **strlwr()**

- A função **strupr()** converte uma string em maiúsculas.
- A função **strlwr()** converte uma string em minúsculas.
- Obs: estas funções não fazem parte da biblioteca padrão da linguagem C, portanto elas funcionam apenas no SO Windows.

Função strlen()

- Retorna o tamanho de uma string.

- Sintaxe:

- `tam = strlen(string);`

- Exemplo:

```
int main(){
    int i, tam;
    char nome[15];
    printf("Digite o nome: \n");
    fgets(nome,sizeof(nome),stdin);
    tam=strlen(nome);
    printf("A palavra %s tem %i caracteres", nome, tam);
}
```

Funções toupper() e tolower()

- A função toupper() recebe como parâmetro um caracter e retorna sua versão em maiúsculo enquanto a função tolower() retorna a versão em minúsculo.

Exemplo toupper()

```
# include <stdio.h>
# include <ctype.h>
int main(){
    int i;
    char nome[15];
    printf("Digite o nome: \n");
    fgets(nome,15,stdin);
    i=0;
    while (nome[i] != '\0'){
        nome[i]=toupper(nome[i]);
        i++;
    }
    printf("\n %s",nome);
    return 0;
}
```

Exemplo tolower()

```
# include <stdio.h>
# include <ctype.h>
int main(){
    int i;
    char nome[15];
    printf("Digite o nome: \n");
    fgets(nome,15,stdin);
    i=0;
    while (nome[i] != '\0'){
        nome[i]=tolower(nome[i]);
        i++;
    }
    printf("\n %s",nome);
    return 0;
}
```

Função strcat()

- Concatena a segunda string no final da primeira string.
- Exemplo:

```
int main(){
    char nome[30], sobrenome[15];
    printf("\n Digite o nome: ");
    fgets(nome,sizeof(nome),stdin);
    printf("\n Digite o sobrenome: ");
    fgets(sobrenome,sizeof(sobrenome),stdin);
    strcat(nome," ");
    strcat(nome,sobrenome);
    printf("\n %s, nome);
    return 0;
}
```

Função `strcspn`

- A função `strcspn` percorre uma `string1` e conta quantos caracteres seguidos não pertencem a `string2`. Quando ela encontra o primeiro caractere em `string1` que está presente em `string2`, ela para e retorna a posição desse caractere (como um índice).

Resolvendo o problema \n no strcat

- Exemplo:

```
int main(){
    char nome[30], sobrenome[15];
    printf("\n Digite o nome: ");
    fgets(nome,sizeof(nome),stdin);
    printf("\n Digite o sobrenome: ");
    fgets(sobrenome,sizeof(sobrenome),stdin);
    nome[strcspn(nome, "\n")] = '\0';
    strcat(nome," ");
    strcat(nome,sobrenome);
    printf("\n %s, nome);
    return 0;
}
```

Problema com scanf e (gets e fgets)

```
int main(){
    int i, idade;
    char nome[15];
    printf("Digite 5 nomes e idades \n");
    for(i=0; i < 5; i++){
        printf("\n Digite o nome:");
        fgets(nome,sizeof(nome),stdin);
        printf("\n Digite a idade:");
        scanf("%d",&idade);
    }
    return 0;
}
```

Problema com scanf e (gets e fgets)

- Quando usamos scanf, o programa salva o que foi digitado em um buffer de entrada, e depois grava o que estiver neste buffer na próxima variável.
- O principal problema com o scanf, é que ele deixa dentro do buffer o "ENTER" que você pressiona para confirmar o que acabou de digitar.
- Então, quando você digita a primeira idade, ele vai "consumir" o número que você digitou, mas vai ficar um ENTER gravado no buffer de entrada; Quando o laço volta para "fgets(nome)", o fgets lê o ENTER que ficou alocado no buffer, e pula esse comando, deixando o nome vazio.
- O programa entende que você não quis colocar nome nenhum, e simplesmente digitou ENTER... Porém esse ENTER você digitou antes), e ele vai direto para o scanf("%d", &idade).



Função `fflush(stdin)` para usar `gets`

- Para solucionar deve ser usado o comando `fflush(stdin)`, para limpar buffer do teclado.
- Ou `getchar()` no visual code

Exemplo com fflush(stdin)

```
int main(){
    int i, idade;
    char nome[15];
    printf("Digite 5 nomes e idades \n");
    for(i=0; i < 5; i++){
        printf("\n Digite o nome:");
        fgets(nome,15,stdin);
        printf("\n Digite a idade:");
        scanf("%d",&idade);
        fflush(stdin);
    }
    return 0;
}
```



Exercício 1

Faça um programa que leia 15 nomes de pessoas e após lidos seja possível pesquisar no vetor se um determinado nome foi digitado. O programa deverá deixar que sejam pesquisados nomes até que seja informado "FIM" no nome.



Exercício 2

- Faça um programa que leia um string e um caracter fornecidos pelo usuário. O programa deverá mostrar na tela quantas vezes o caracter aparece nessa string.



Exercício 3

- Faça um programa que receba uma string, o caracter que será trocado e o novo caracter. O programa deverá trocar um caracter de uma string por outro caracter.



Exercício 4

- Faça um programa que receba uma palavra e verifique se constitui um palíndromo, ou seja, escrita do fim para o começo fica igual escrita do começo para o fim. Ex: RENNEN, ANA, MIRIM



Exercício 5

- Faça um programa para criptografar uma frase dada pelo usuário (trocando as vogais da frase por *).